SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering
ASSESSMENT TOOL 2 – MODEL COMPARISON PRESENTATION (BERT VS GPT VS T5 VS LLAMA)

Course Code:	CSA65	**Course Title:**	Generative AI and Large Language Models
CO Assessed:	CO4 – Precision (BL3, BL4,BL5)	**Assessment:**	Assessment Tool 2 - Model Comparison Presentation (BERT vs GPT vs T5 vs LLaMA)
Weightage:	10% of CIA	**Total Marks:**	2 * 50= 100
CO4 Statement:	*Design and implement multimodal Generative AI applications integrating text, image, and speech models for engineering applications.*(BL3, BL4,BL5)		
Student Name:	C Anu Keerthana	**Reg. No.:**	192472057
Section / Batch:	CSE-AI	**Date:**	21-08-2026

Instructions to Students

- **Answer any 2 questions. Each question carries 50 marks. Total: 100 marks.**
- **Compare all four models: BERT, GPT, T5, and LLaMA.**
- Explain the **architecture and working principle** of each model.
- Compare their **pre-training objectives and capabilities**.
- Map each model to appropriate **real-world applications**.
- Identify the **strengths and limitations** of each model.
- Support the presentation using **credible research papers, official documentation, benchmarks, or other reliable sources**.
- Include at least one **comparison table or diagram**.
- Use relevant **real-world case studies or application examples**.
- Conclude by identifying the **most suitable model for the selected application** and justify the decision.
- Include proper **references/citations** in the presentation.

Code of Conduct

I C Anu Keerthana/192472057 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: C Anu Keerthana

SECTION A : MODEL COMPARISON PRESENTATION (BERT VS GPT VS T5 VS LLAMA)

APPLICATION MAPPING

11. Compare BERT, GPT, T5, and LLaMA for text classification. Identify the most suitable model and justify your choice with real-world examples.

1. Title

FAISS Document-to-Vector Mapping for Semantic Search

2. Objective

The objective of this experiment is to develop a FAISS-based semantic search system where vector indices are mapped to the original document IDs and document contents using a separate data structure. The system performs Top-5 retrieval and displays the corresponding original documents.

3. Technology Used

- Python
- Google Colab
- Sentence Transformers
- FAISS
- NumPy

4. Methodology

A collection of documents is created with unique document IDs and contents. The document contents are converted into vector embeddings using the SentenceTransformer model.

These embeddings are stored in a FAISS index. A separate dictionary is maintained to map each FAISS vector index to its corresponding original document ID and document content.

When a user enters a query, the query is converted into an embedding and searched using FAISS. FAISS returns the most similar vector indices, which are then mapped back to the original documents.

5. Working Process

Original Documents
↓
Assign Document IDs
↓
Create Vector-to-Document Mapping
↓
Generate Document Embeddings
↓
Store Embeddings in FAISS
↓
User Query
↓

Generate Query Embedding

↓

FAISS Top-5 Search

↓

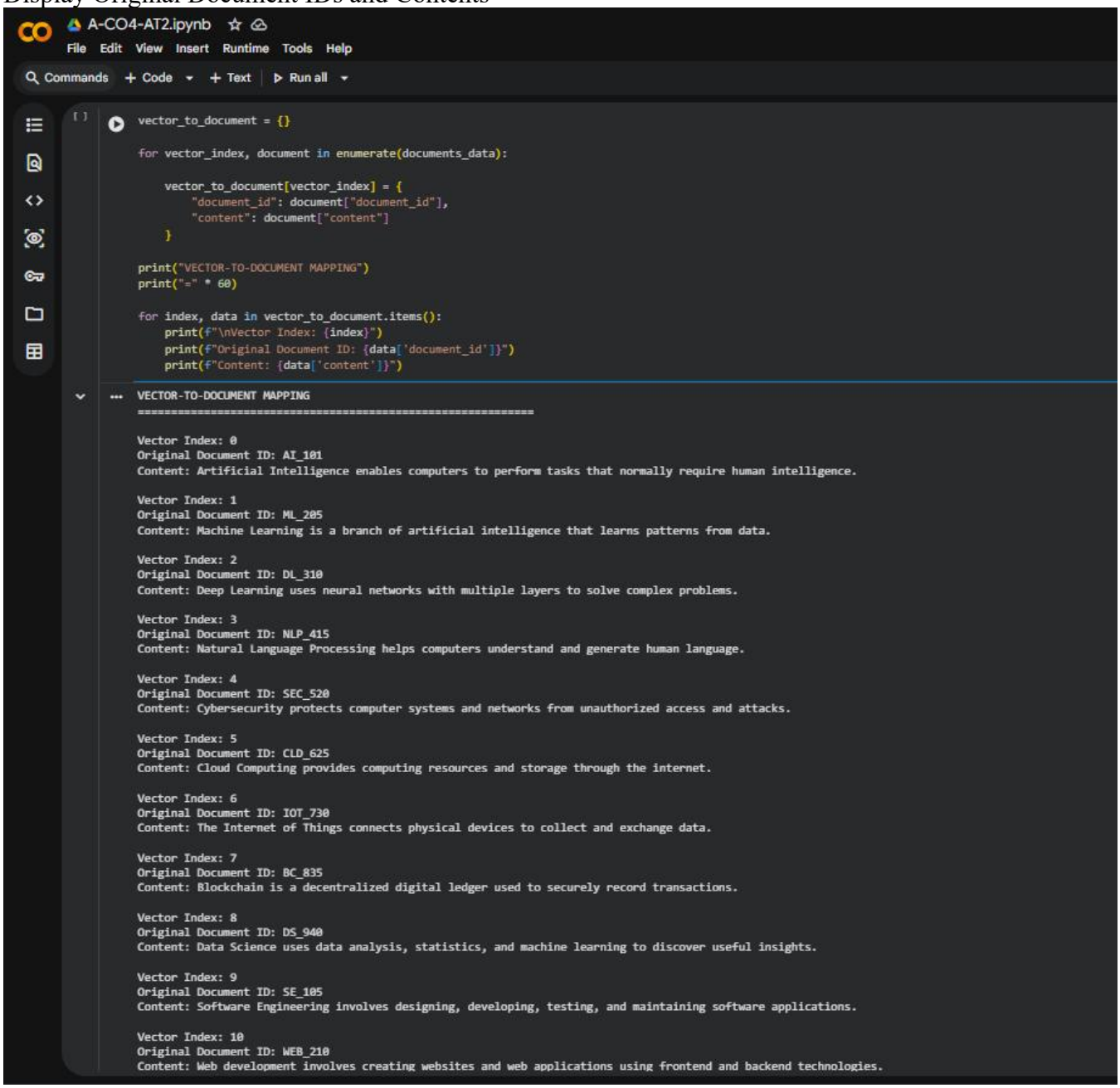
Vector Indices Retrieved

↓

Mapping Dictionary

↓

Display Original Document IDs and Contents

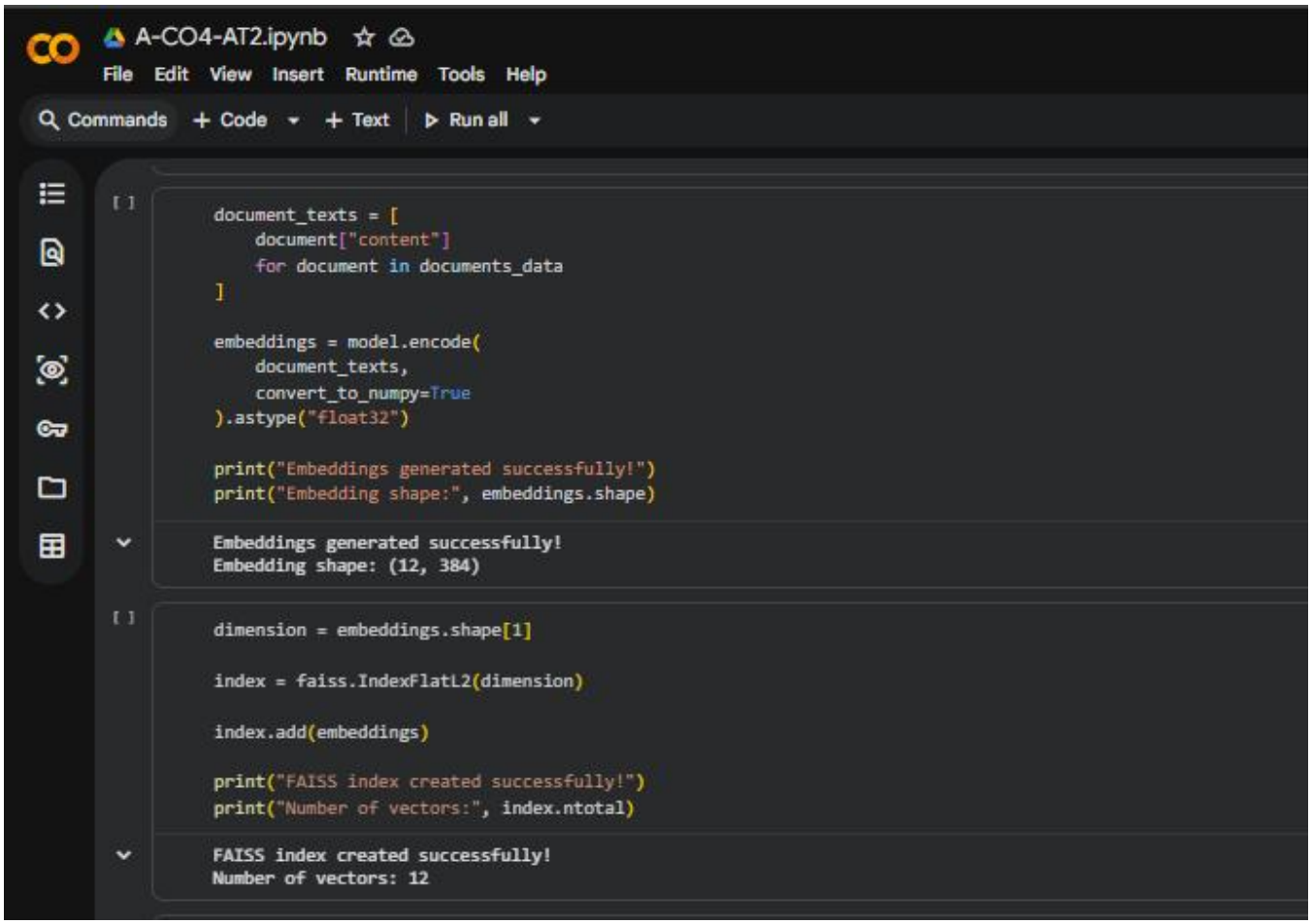


6. Implementation

The application was implemented using the following steps:

1. Created a collection of documents with unique IDs.
2. Stored document IDs and contents in a Python data structure.

3. Created a separate vector-to-document mapping dictionary.
4. Loaded the SentenceTransformer embedding model.
5. Generated embeddings for all document contents.
6. Created a FAISS IndexFlatL2 vector index.
7. Added document embeddings to the FAISS index.
8. Converted the user query into a vector embedding.
9. Performed Top-5 semantic retrieval.
10. Used the mapping dictionary to display the original document ID and content.



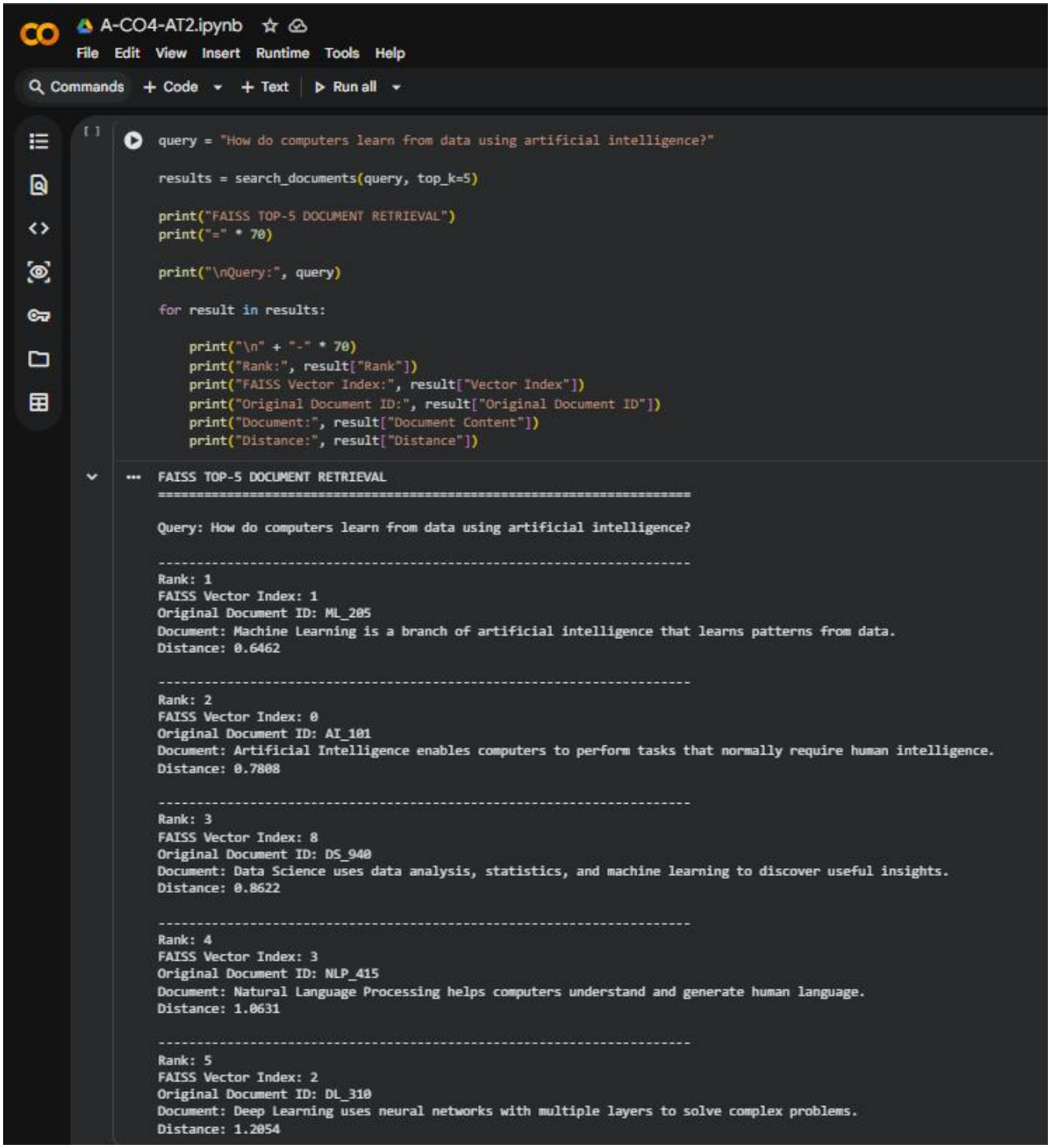
7. Sample Input

Example query:

How do computers learn from data using artificial intelligence?

Other test queries used include:

How does machine learning learn from data?
How can computers understand human language?
What technology protects systems from cyber attacks?



8. Output

For each retrieved result, the system displays:

- Rank
- FAISS Vector Index
- Original Document ID
- Document Content
- Distance Value

Example:

Rank: 1
Vector Index: 1
Original Document ID: ML_205
Document: Machine Learning is a branch of artificial intelligence that learns patterns from data.
Distance: 0.XXXX

The exact distance values and ranking may vary depending on the generated embeddings.

9. Importance of Vector-to-Document Mapping

FAISS stores numerical vector embeddings and returns the position or index of the retrieved vector. The vector index alone does not provide meaningful information to the user.

Therefore, a separate mapping is required:

Vector Index
↓
Original Document ID
↓
Original Document Content

For example:

Vector Index: 0

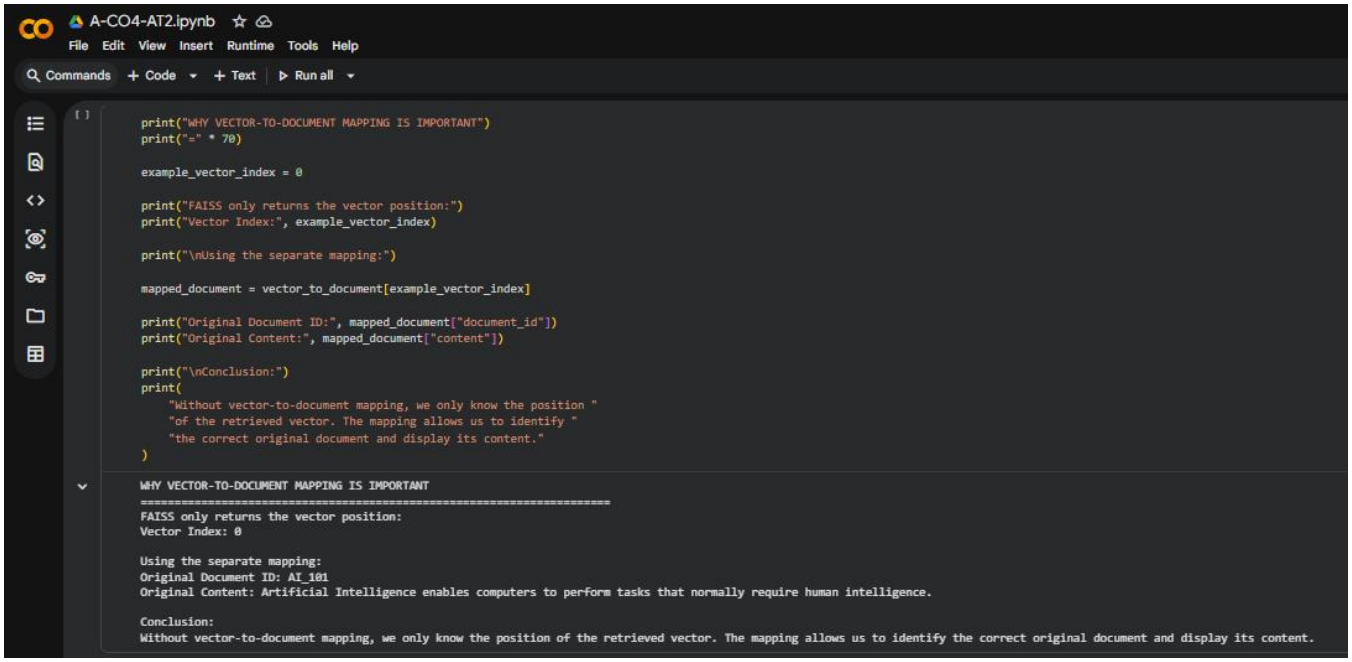
↓ Mapping

Document ID: AI_101

↓ Mapping

Artificial Intelligence enables computers to perform
tasks that normally require human intelligence.

This ensures that the correct original document is displayed after FAISS retrieval.



10. Conclusion

The FAISS Document-to-Vector Mapping system was successfully implemented using Python, Sentence Transformers, and FAISS. Document embeddings were stored in a FAISS index, while a separate mapping structure maintained the relationship between vector indices, original document IDs, and document contents.

The system successfully performed Top-5 semantic retrieval and displayed the corresponding original documents. Maintaining correct vector-to-document mapping is essential because FAISS returns vector positions, and the mapping allows those positions to be connected back to meaningful document information.

12. Compare BERT, GPT, T5, and LLaMA for text summarization. Explain which model or architecture is most appropriate and why.

1. Title

Performance Comparison of Individual and Batch Query Search Using FAISS

2. Objective

The objective of this experiment is to implement individual-query and batch-query search using FAISS. The system processes at least 20 queries and compares the total execution time and average query latency of both approaches.

3. Technologies Used

- Python
- Google Colab
- FAISS

- Sentence Transformers
- NumPy
- Pandas

4. Methodology

A collection of 1,000 sample documents was created covering different technology topics such as Artificial Intelligence, Machine Learning, Deep Learning, Natural Language Processing, Cybersecurity, Cloud Computing, and Blockchain.

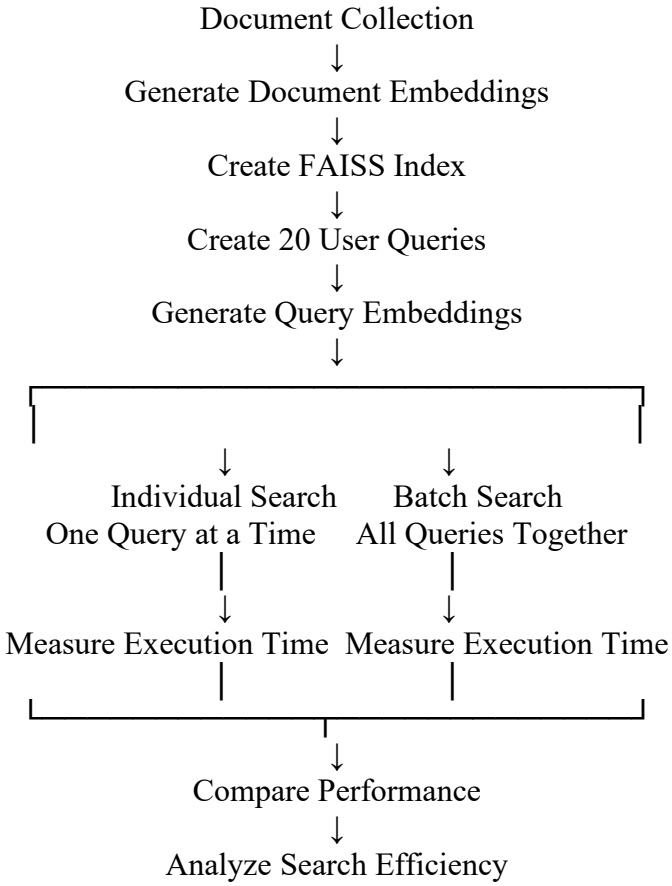
The documents were converted into vector embeddings using the all-MiniLM-L6-v2 Sentence Transformer model. These embeddings were stored in a FAISS index for similarity search.

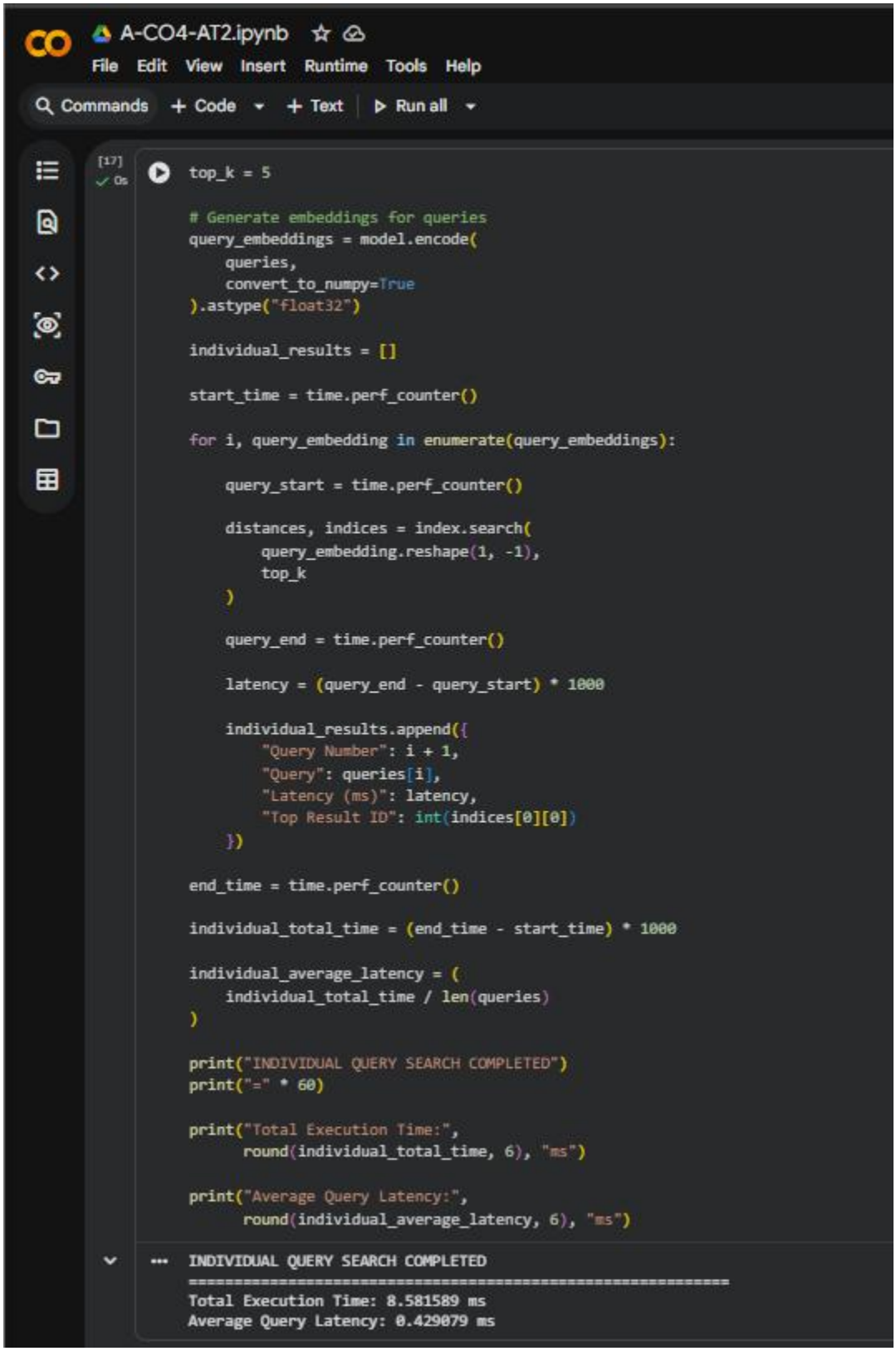
A set of 20 different user queries was then created. The queries were processed using two approaches:

1. Individual Query Search – Each query was searched separately.
2. Batch Query Search – All 20 query embeddings were sent to FAISS together.

The execution time and average query latency of both approaches were measured and compared.

5. Working Process



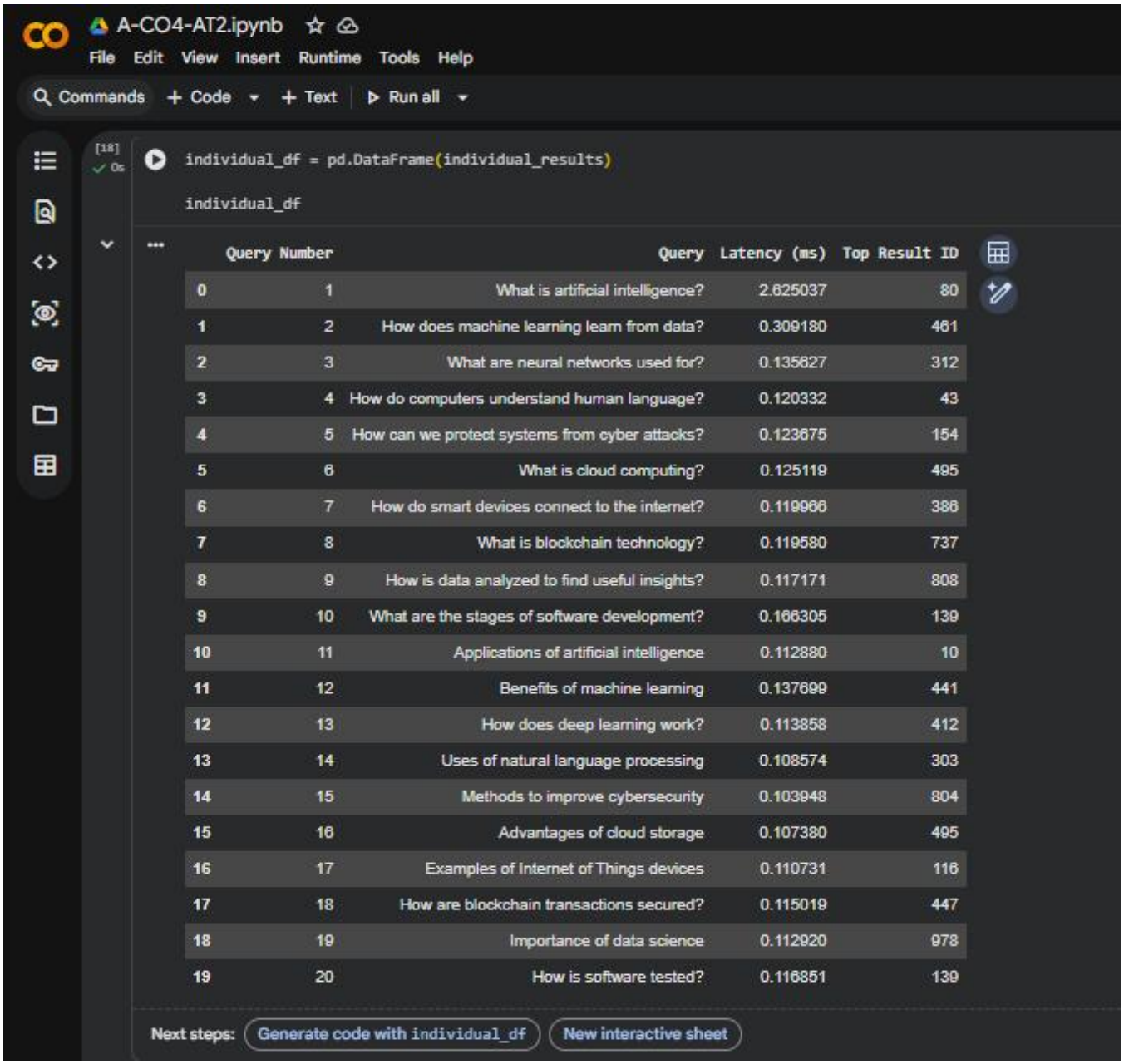


6. Implementation

The application was implemented using the following steps:

1. Created a dataset containing 1,000 documents.
2. Loaded the Sentence Transformer embedding model.

3. Converted all documents into numerical vector embeddings.
4. Created a FAISS vector index and stored the document embeddings.
5. Created 20 different semantic search queries.
6. Converted all queries into vector embeddings.
7. Performed individual search by processing one query at a time.
8. Recorded the total execution time and latency.
9. Performed batch search by processing all queries together.
10. Compared the performance of both methods.
11. Verified that both approaches returned consistent retrieval results.



	Query Number	Query	Latency (ms)	Top Result ID
0	1	What is artificial intelligence?	2.625037	80
1	2	How does machine learning learn from data?	0.309180	461
2	3	What are neural networks used for?	0.135627	312
3	4	How do computers understand human language?	0.120332	43
4	5	How can we protect systems from cyber attacks?	0.123675	154
5	6	What is cloud computing?	0.125119	495
6	7	How do smart devices connect to the internet?	0.119966	386
7	8	What is blockchain technology?	0.119580	737
8	9	How is data analyzed to find useful insights?	0.117171	808
9	10	What are the stages of software development?	0.166305	139
10	11	Applications of artificial intelligence	0.112880	10
11	12	Benefits of machine learning	0.137699	441
12	13	How does deep learning work?	0.113858	412
13	14	Uses of natural language processing	0.108574	303
14	15	Methods to improve cybersecurity	0.103948	804
15	16	Advantages of cloud storage	0.107380	495
16	17	Examples of Internet of Things devices	0.110731	116
17	18	How are blockchain transactions secured?	0.115019	447
18	19	Importance of data science	0.112920	978
19	20	How is software tested?	0.116851	139

7. Test Inputs

The system was tested using multiple queries, including:

What is artificial intelligence?

How does machine learning learn from data?

What are neural networks used for?

How do computers understand human language?

How can we protect systems from cyber attacks?

What is cloud computing?

How do smart devices connect to the internet?

What is blockchain technology?

How is data analyzed to find useful insights?

What are the stages of software development?

Additional queries were used to make a total of **20 test cases**.

A-CO4-AT2.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
BATCH QUERY SEARCH COMPLETED
============================================================
Total Execution Time: 5.968442 ms
Average Query Latency: 0.298422 ms
```

[28] ✓ 0s

```python
batch_results = []

for i in range(len(queries)):

    batch_results.append({
        "Query Number": i + 1,
        "Query": queries[i],
        "Top Result ID": int(batch_indices[i][0]),
        "Top-5 Document IDs": list(
            map(int, batch_indices[i])
        )
    })

batch_df = pd.DataFrame(batch_results)

batch_df
```

	Query Number	Query	Top Result ID	Top-5 Document IDs
0	1	What is artificial intelligence?	80	[80, 440, 420, 390, 690]
1	2	How does machine learning learn from data?	461	[461, 441, 381, 271, 331]
2	3	What are neural networks used for?	312	[312, 402, 282, 412, 192]
3	4	How do computers understand human language?	43	[43, 393, 313, 273, 423]
4	5	How can we protect systems from cyber attacks?	154	[154, 374, 484, 364, 584]
5	6	What is cloud computing?	495	[495, 395, 215, 485, 335]
6	7	How do smart devices connect to the internet?	386	[386, 396, 326, 486, 596]
7	8	What is blockchain technology?	737	[737, 197, 357, 267, 747]
8	9	How is data analyzed to find useful insights?	808	[808, 328, 858, 868, 128]
9	10	What are the stages of software development?	139	[139, 179, 339, 209, 609]
10	11	Applications of artificial intelligence	10	[10, 390, 80, 480, 380]
11	12	Benefits of machine learning	441	[441, 281, 521, 191, 451]
12	13	How does deep learning work?	412	[412, 142, 312, 242, 112]
13	14	Uses of natural language processing	303	[303, 223, 343, 233, 283]
14	15	Methods to improve cybersecurity	804	[804, 484, 384, 604, 644]
15	16	Advantages of cloud storage	495	[495, 215, 395, 185, 335]
16	17	Examples of Internet of Things devices	116	[116, 386, 296, 106, 36]
17	18	How are blockchain transactions secured?	447	[447, 317, 457, 7, 557]
18	19	Importance of data science	978	[978, 198, 338, 218, 278]
19	20	How is software tested?	139	[139, 179, 299, 529, 79]

8. Individual Query Search

In individual query search, each query is processed separately.

For every query:

1. The query embedding is selected.
2. The query is sent individually to the FAISS index.
3. FAISS retrieves the Top-5 most similar documents.
4. The search latency is recorded.
5. The process is repeated for all 20 queries.

The total execution time is calculated after processing all queries.

9. Batch Query Search

In batch query search, all query embeddings are processed together in a single FAISS search operation.

The process is:

1. All 20 queries are converted into embeddings.
2. The complete set of query embeddings is sent to FAISS.
3. FAISS retrieves the Top-5 similar documents for every query.
4. The total execution time is recorded.
5. The average latency per query is calculated.

Batch processing reduces the need to repeatedly call the search operation for every individual query.

10. Performance Comparison

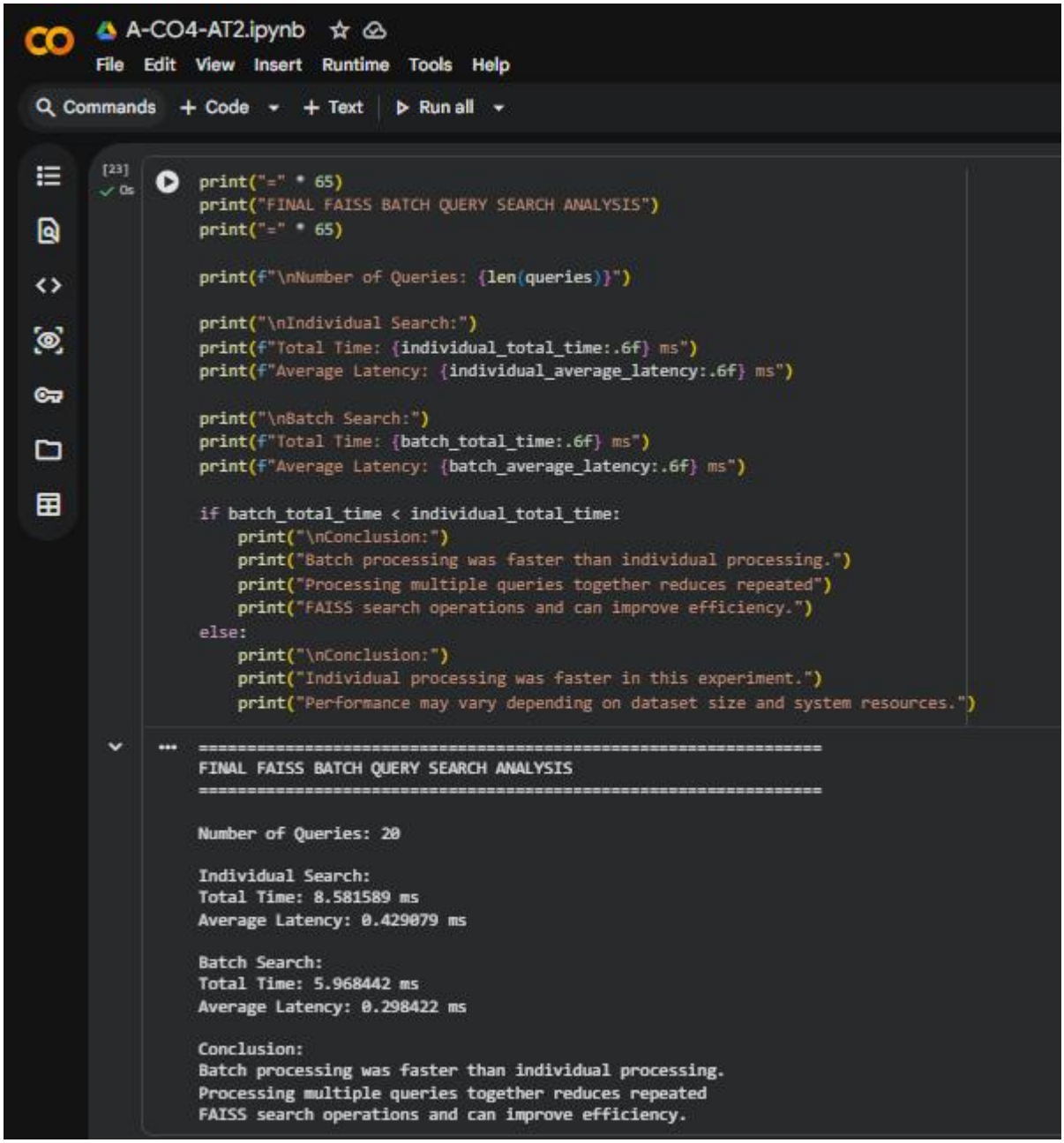
The following parameters were compared:

Search Method	Number of Queries	Total Execution Time	Average Query Latency
Individual Query Search	20	Measured during execution	Calculated
Batch Query Search	20	Measured during execution	Calculated

Note: Insert your actual output values from Google Colab into this table.

The average query latency was calculated using:

Average Query Latency =
Total Execution Time / Number of Queries



11. Result and Conclusion

The FAISS Batch Query Search system was successfully implemented using Python, Sentence Transformers, and FAISS. A collection of 1,000 documents was converted into vector embeddings and stored in a FAISS index.

Twenty different queries were tested using both individual-query search and batch-query search. The total execution time and average query latency were measured for both approaches. The retrieved results were also checked to ensure consistency.

Batch processing can provide better efficiency because multiple query vectors are processed together rather than repeatedly calling the search operation for each individual query. However, the actual performance difference may depend on the dataset size, number of queries, available hardware, and system resources.

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Architectural Comparison Accuracy	30%	Accurate, in-depth comparison of architectures and use-cases	Accurate comparison with minor gaps	Superficial comparison of models	Inaccurate or confused comparison
Application Mapping	25%	Clearly maps each model to appropriate real-world applications	Reasonable mapping to applications	Weak mapping to applications	No meaningful mapping presented
Research Depth	20%	Well-researched with credible sources/benchmarks cited	Adequately researched	Limited research depth	Unresearched / opinion-based content
Delivery	15%	Confident, engaging, well-paced delivery	Clear delivery with minor pacing issues	Hesitant delivery	Unclear or unprepared delivery
Slide Design & References	10%	Well-designed slides with proper references	Adequate slides, minor reference gaps	Basic slides, poor referencing	No slides or references provided

Marks Evaluation:

Question No.	Architectural Comparison Accuracy (30%)	Application Mapping (25%)	Research Depth (20%)	Delivery (15%)	Slide Design & References (10%)	Total Marks (100)
Q1						
Q2						
					Overall Total (100)	